

Make “implicit expression variations” more explicit (Wording for US185)

Document #: P2102R0
Date: 2020-02-11
Project: Programming Language C++
LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>
Casey Carter
<casey@carter.net>

§ 1 Introduction

This paper provides wording to resolve NB comment [US185](#).

§ 2 Wording

This wording is relative to [\[N4849\]](#).

1. Edit 18.2 [\[concepts.equality\]](#)p1 as indicated to define “operand”:

An expression is *equality-preserving* if, given equal inputs, the expression results in equal outputs. The inputs to an expression are the set of the expression’s operands. The output of an expression is the expression’s result and all operands modified by the expression. For the purposes of this subclause, the *operands* of an expression are the largest subexpressions that include only:

- (1.1) — an *id-expression* (7.5.4 [\[expr.prim.id\]](#)), and
- (1.2) — invocations of the library function templates `std::move`, `std::forward`, and `std::declval` (20.2.4 [\[forward\]](#), 20.2.6 [\[declval\]](#))

[*Example*: The operands of the expression `a = std::move(b)` are `a` and `std::move(b)`. — *end example*]

2. Edit 18.2 [\[concepts.equality\]](#) p5 to clarify how the cv-qualification and value categories of operands are determined:

An expression that may alter the value of one or more of its inputs in a manner observable to equality-preserving expressions is said to modify those inputs. This document uses a notational convention to specify which expressions declared in a *requires-expression* modify which inputs: except where otherwise specified, an expression operand that is a non-constant lvalue or rvalue may be modified. Operands that are constant lvalues or rvalues are required

to not be modified. For the purposes of this subclause, the cv-qualification and value category of each operand is determined by assuming that each template type parameter denotes a cv-unqualified complete non-array object type.

3. Edit 18.6 [\[concepts.object\]](#) to explicitly require additional variations on assignable_from for copyable:

```
template<class T>
- concept copyable = copy_constructible<T> && movable<T> &&
    assignable_from<T&, const T&>;
+ concept copyable = copy_constructible<T> && movable<T> &&
    assignable_from<T&, T&> &&
+    assignable_from<T&, const T&> && assignable_from<T&,
    const T>;
```

4. Edit 23.3.7.3 [\[alg.req.ind.copy\]](#) to explicit require additional variations on indirectly_writable for indirectly_copyable_storable:

```
template<class In, class Out>
    concept indirectly_copyable_storable =
        indirectly_copyable<In, Out> &&
+    indirectly_writable<Out, iter_value_t<In>&> &&
    indirectly_writable<Out, const iter_value_t<In>&> &&
+    indirectly_writable<Out, iter_value_t<In>&&> &&
+    indirectly_writable<Out, const iter_value_t<In>&&> &&
    copyable<iter_value_t<In>> &&
    constructible_from<iter_value_t<In>, iter_reference_t<In>> &&
    assignable_from<iter_value_t<In>&, iter_reference_t<In>>;
```

§ 3 References

[N4849] Richard Smith. 2020. Working Draft, Standard for Programming Language C++. <https://wg21.link/n4849>